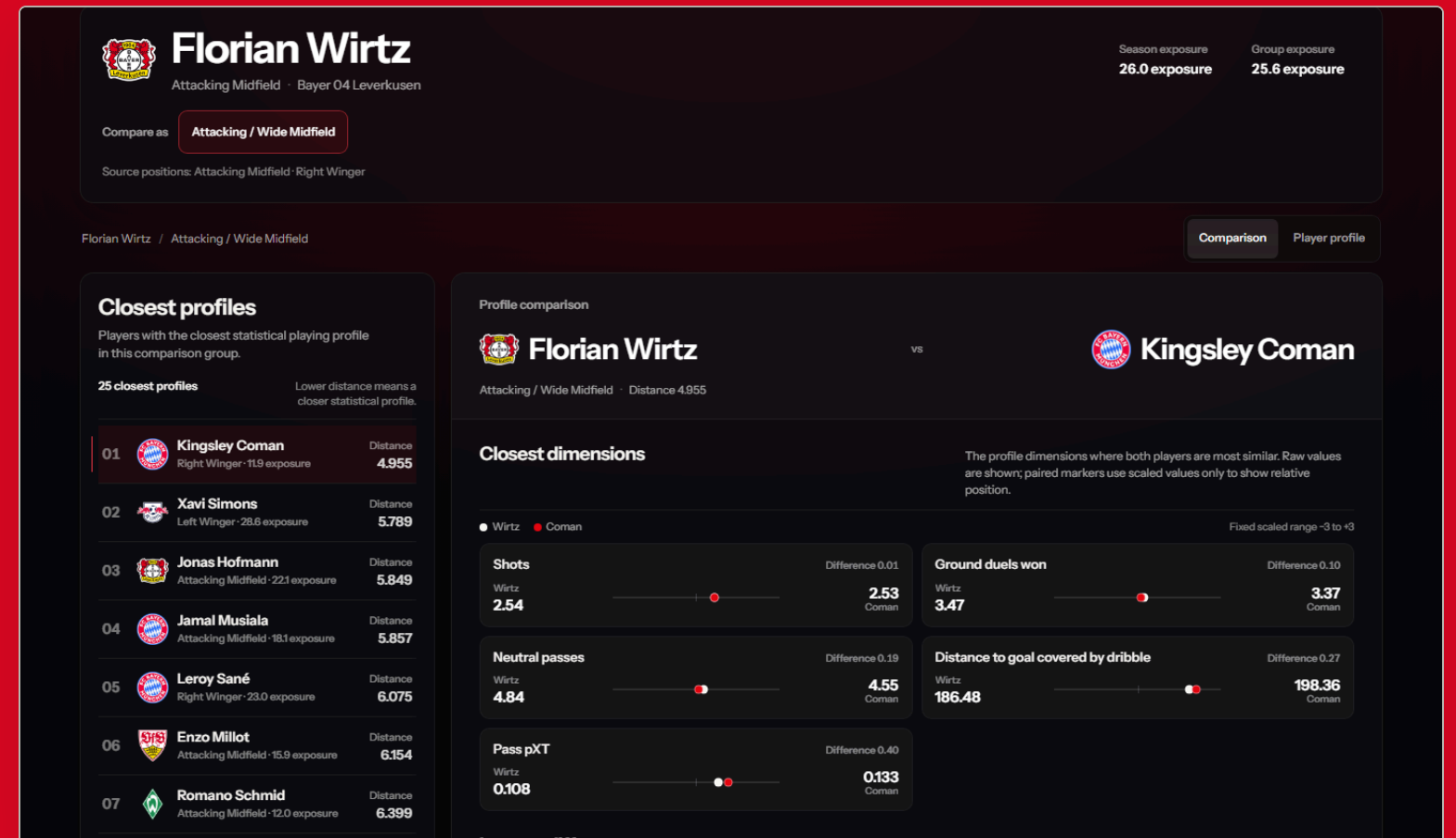




Player Replacement

An explainable search for players with a similar statistical profile.

Built from Bundesliga 2023/24 IMPECT Open Data.



Who

closest profiles

Why

closest dimensions

Trade-off

largest differences

Rafa Agustant

Data analytics / Product design / Full-stack build

The replacement problem

Raw totals do not tell us whether two players occupy a similar role.

If a player leaves, who plays in a similar way?

A passing total means something different for a centre back and a winger. Exposure and metric scale also change the comparison.

I wanted the shortlist to be inspectable. A name and a distance value are not enough on their own.

A useful comparison keeps

01 Context

Players stay inside the same broad comparison group.

02 Exposure

Additive actions are normalized by the exposure in that group.

03 Explanation

The output shows where two profiles align and where they differ.

This is decision support, not a player score.

Why I froze Touchline

The technical work was useful. The product question was still too broad.

Touchline

A general football exploration layer

- Proved the data pipeline and interface could work
- Left too many possible directions for the user
- Adding more screens would not sharpen the purpose

Narrow the question



Player Replacement

One clear question

Who has the closest playing profile, and what explains the match?

- One recruitment-oriented decision
- One result that can be checked and challenged

The smaller scope gave every analysis step a clear job.

What the data actually supports

IMPECT Open Data / Bundesliga 2023/24

306



matches

18



teams

494



KPI-observed players

101



KPI fields reviewed

Included in V1

- Player-match-position KPI records across ten source positions
- Exposure represented through matchShare

Intentionally excluded

- Goalkeeper comparison
- Transfer feasibility, market data, and player-quality ratings



I kept V1 inside the decisions this dataset could support.

Independent project. No official league affiliation.

The audit changed the method

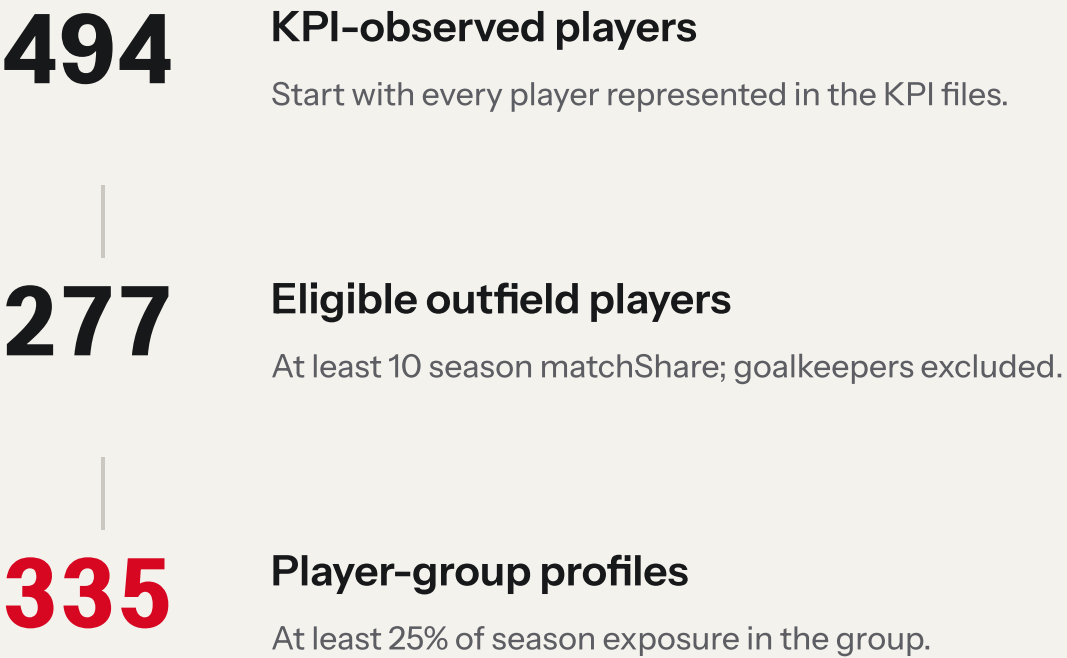
Three source behaviors mattered before I calculated any similarity.

What I found		What it changed
01	Exposure playDuration follows long source timelines and can exceed normal minute expectations.	→ Use matchShare as full-match-equivalent exposure.
02	Position Many players had sequential spells in several source positions.	→ Map source positions to five groups and keep multi-group membership.
03	Missingness Every KPI had gaps, including ten fields at least 90% missing.	→ Do not treat every missing value as zero.

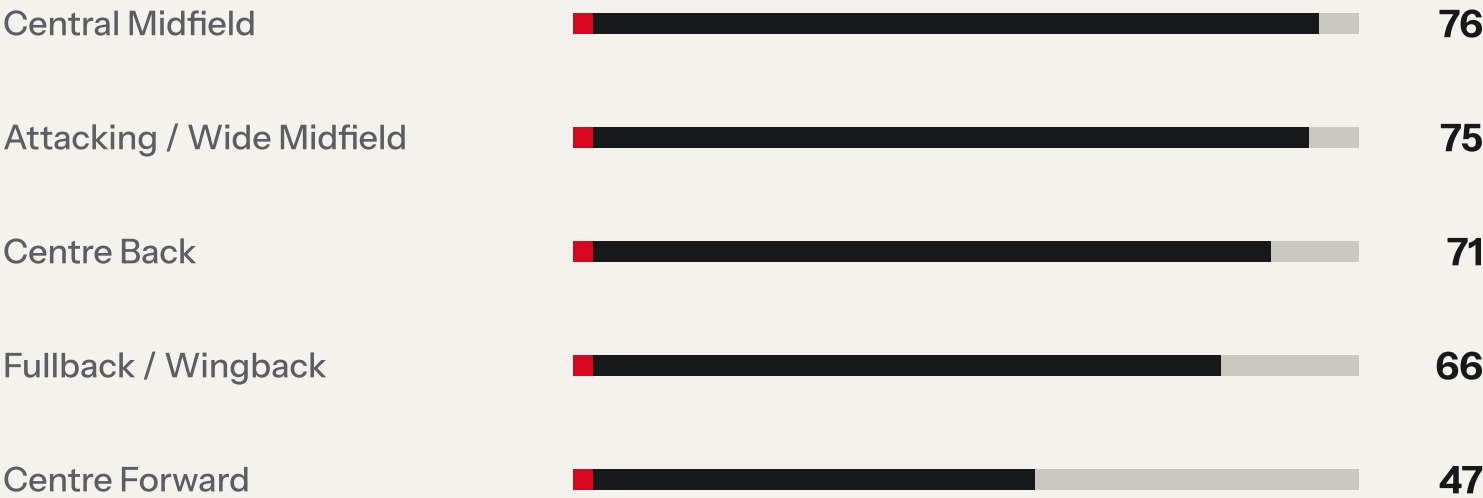
Defining who could be compared

The first challenge was deciding which player profiles the data could support.

Profile construction



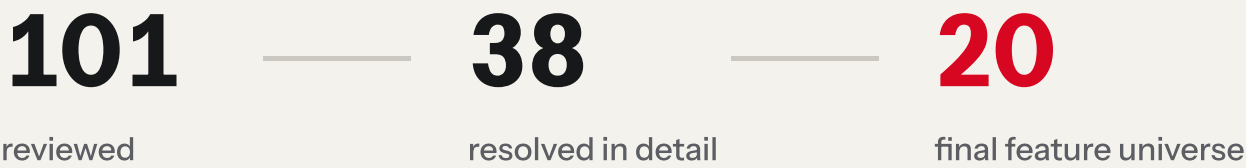
Profiles by comparison group



A player can have a different profile in each valid group.

Reducing 101 KPIs to a usable feature set

I kept metrics with clear meaning, usable coverage, and a defensible aggregation rule.



22 initial keeps + 16 review cases

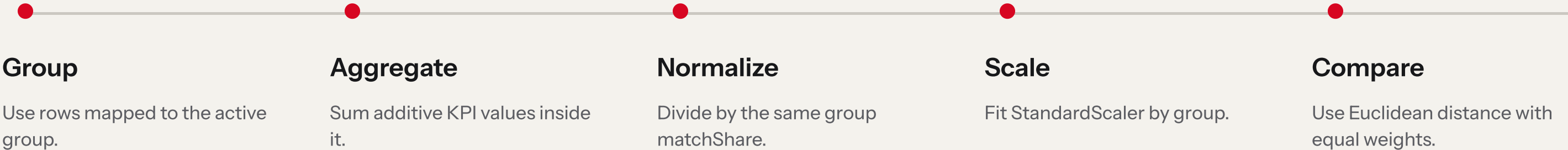
16 common features

Passing	4	Defending	2
Progression	4	Duels	2
Ball involvement	3	Attacking	1

Small position-specific layer		
Centre Back	Aerial duels won / lost	18
Fullback / Wingback	No additions	16
Central Midfield	No additions	16
Attacking / Wide Midfield	Shots / shot xG	18
Centre Forward	Both additions	20

How the similarity search works

Every step can be traced back to the source data and a feature difference.



Season profile

$$\text{sum(KPI values in group)} / \text{sum(matchShare in group)}$$

Candidate rule

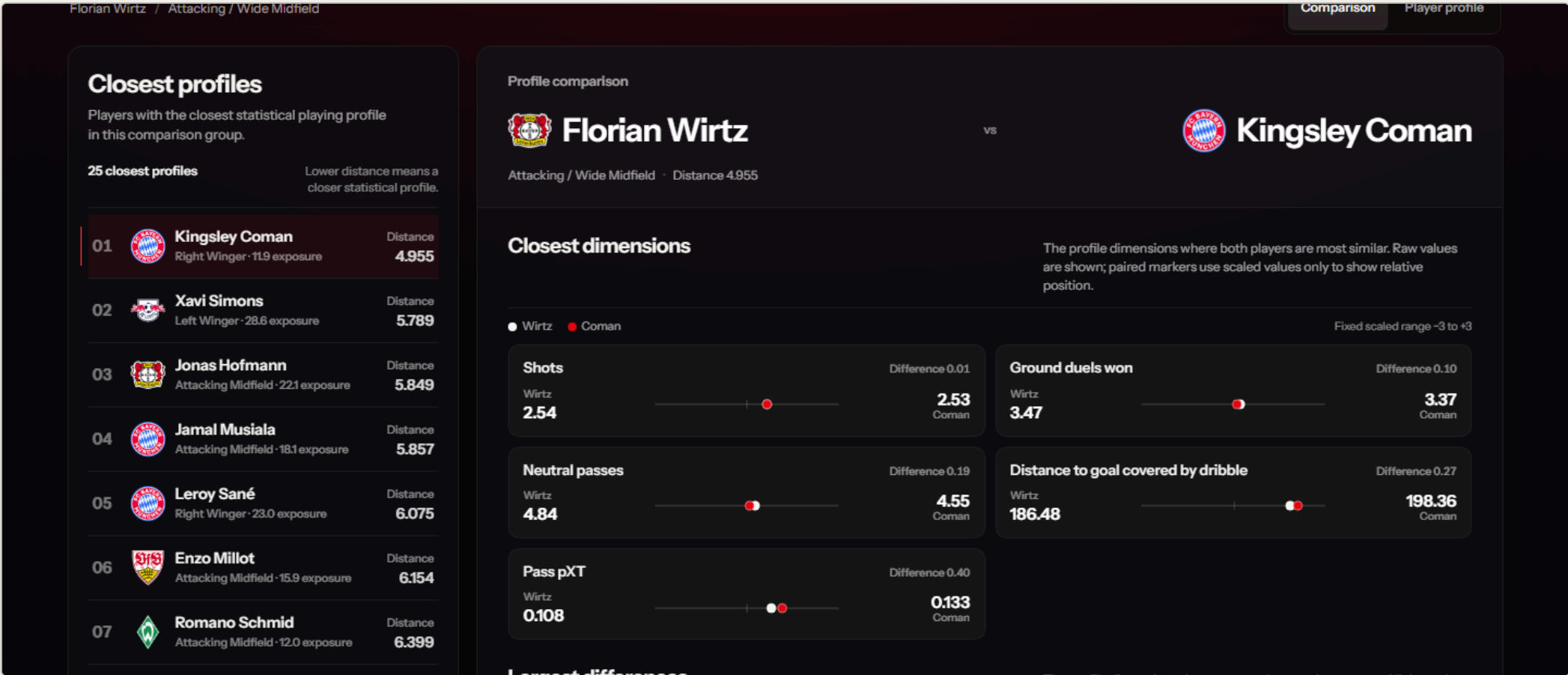
Same group; group matchShare >= 5
The query may have lower exposure.

Lower distance means a closer statistical profile.

I kept equal weights because the data did not justify claiming one feature mattered more.

Who is close, why, and where they differ

A ranking is only useful if the user can inspect it.



01 Who

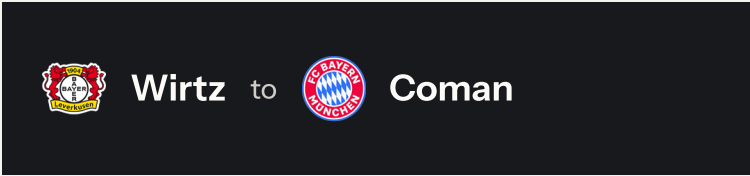
Closest profiles inside the selected comparison group.

02 Why

The dimensions with the smallest scaled differences.

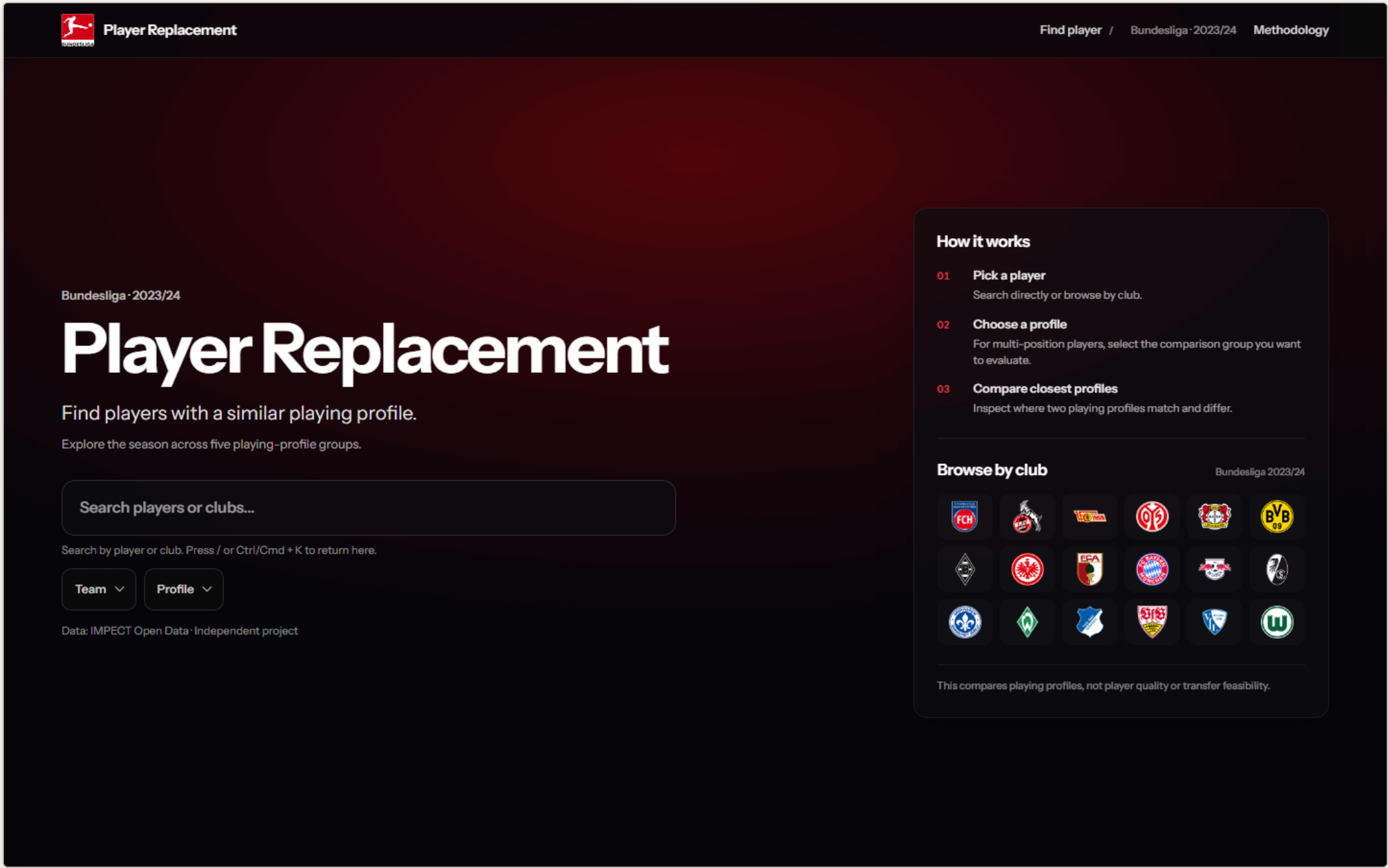
03 Trade-off

The largest differences, with raw values and direction visible.



How users start the search

Search by player, browse a club, or filter by comparison profile.



Three entry points

- 01

Search a player

Start with a name when the target is already known.
- 02

Browse a club

Use the 18 Bundesliga 2023/24 crests to discover players.
- 03

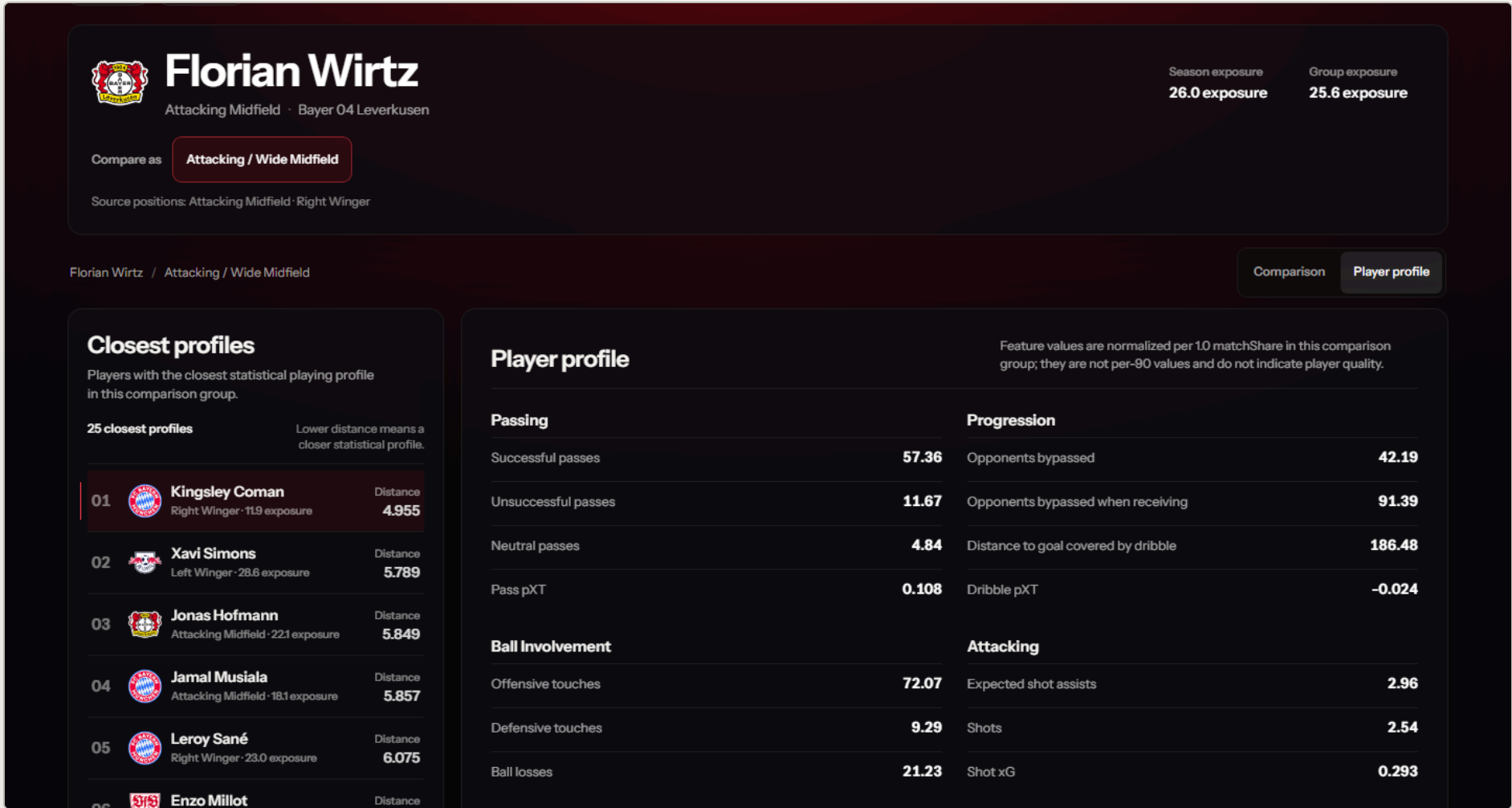
Filter a profile

Browse one valid comparison group without changing the ranking method.

The entry point stays simple even when the user does not know a player name.

Turning the method into a product

FastAPI serves the frozen search logic; React handles discovery and comparison.



Application layer

01 Processed profiles

Parquet tables and frozen feature specifications.

02 FastAPI

Loads assets and fits one scaler per comparison group once.

03 React + Vite

Handles discovery, URL state, search, and feature-level explanation.

Every request uses the same frozen feature and scaling rules.



BUNDESLIGA

What worked, and what I would test next

01 Data decisions were documented

Exposure, position, and missing-value rules came from provider semantics.

02 The ranking stayed inspectable

Feature reduction, group scaling, and distance remain visible.

03 Positional context stayed intact

Multi-position players get a separate profile in each valid group.

The hardest part was deciding what a fair comparison should look like.

Next test

Run the same method on another league or season before adding more features.

That would show whether the comparison rules travel beyond one dataset.

Current boundary

One league, one season, outfield players, and no transfer-feasibility claim.